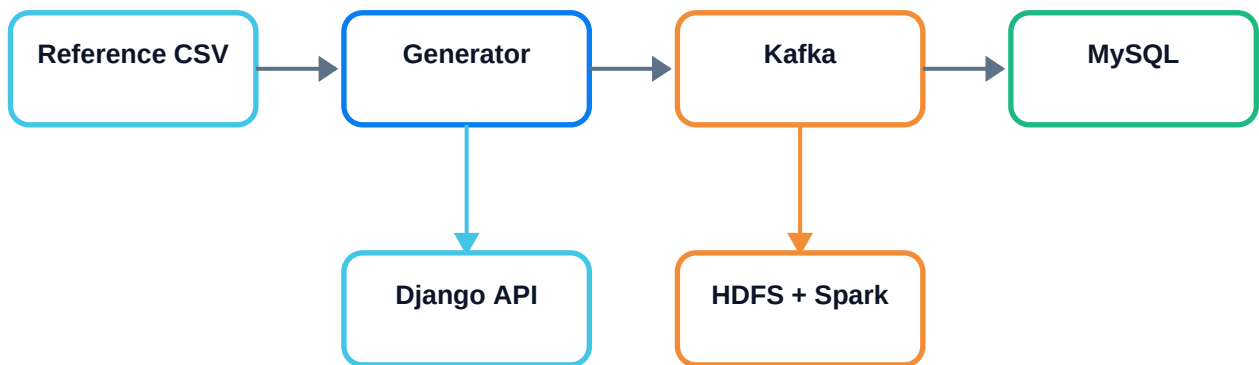


Kigali Public Transport Delay Analytics Platform

Project Report | Big Data Essentials Final Project



Live operational path

A practical big data pipeline for generating transport events, streaming them through Kafka, storing them in MySQL, and supporting delay decisions through a Django dashboard and Spark MLlib.

Prepared for academic demonstration and local Windows deployment

1. Executive Summary

This project implements an end-to-end transport intelligence platform for Kigali. It starts with a historical Rwanda public transport delay dataset and turns its empirical relationships into a continuous stream of synthetic trip events. The events can be generated directly into MySQL or published through Kafka for streaming demonstrations.

The operational layer is a Django dashboard backed by MySQL. It reports total trips, observed delay rate, active routes, transport mix, recent trip activity, map coordinates, and a feature-based delay risk estimate. A separate HDFS and Spark MLlib path demonstrates how the same data domain can support distributed storage and model training.

The central design decision is to use MySQL as the single runtime operational source of truth. Historical SQLite records are treated only as a migration source; live dashboard and generator writes go to MySQL.

Project objectives

- Create realistic synthetic trip events while preserving relationships in the reference data.
- Demonstrate route-keyed Kafka publishing and broker acknowledgement.
- Provide a queryable MySQL operational store for live dashboard metrics.
- Show a Hadoop and Spark MLlib workflow without leaking target variables into model features.

2. Problem Definition and Scope

Urban transport operations generate data across routes, vehicles, schedules, weather conditions, traffic levels, and observed delays. A useful platform must bring these signals together quickly enough for an operator to identify high-risk routes and understand current network conditions.

The project focuses on the data engineering lifecycle rather than a production dispatch system. It uses synthetic events derived from historical observations, which makes the system safe to demonstrate and repeatable on a local Windows machine.

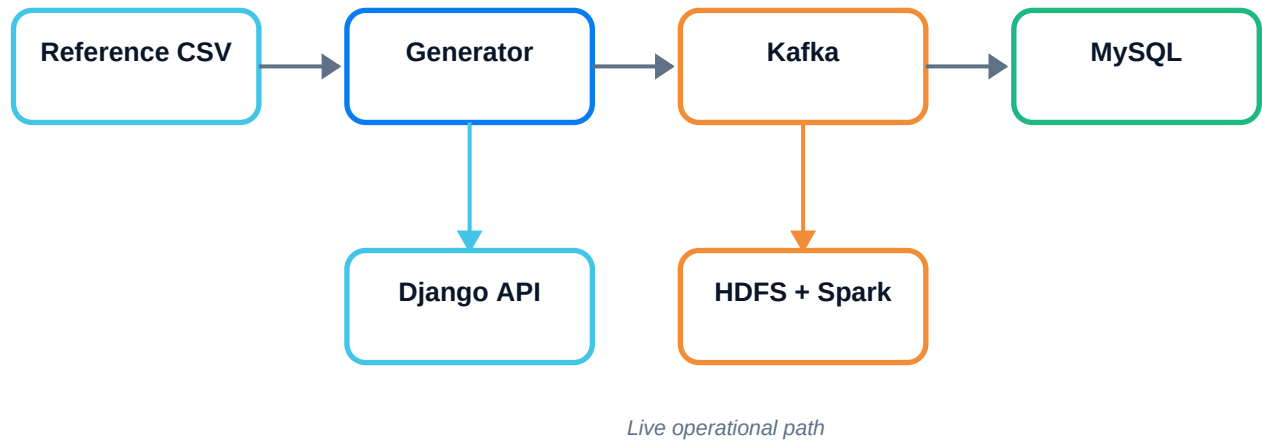
Scope boundaries

Included	Purpose
Synthetic event generation	Create repeatable live-like records from the seed distribution.
Kafka publishing	Move keyed events through a streaming topic.
MySQL persistence	Support operational queries, replay, and dashboard summaries.
Django REST dashboard	Expose current metrics, map data, and risk estimates.
HDFS and Spark MLlib	Demonstrate historical storage and distributed model training.

The system is intentionally modular: generation, transport, storage, web presentation, and model training can be inspected independently while still forming one coherent pipeline.

3. System Architecture

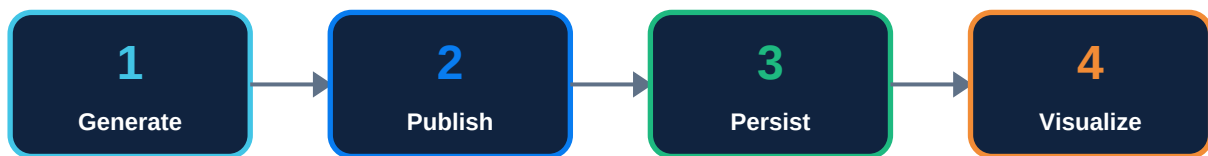
The architecture separates the live operational path from the historical analytics path. The live path prioritizes low-latency generation, streaming, and dashboard queries. The analytics path prioritizes durable historical storage and model training.



Component	Responsibility	Implementation
Reference data	Empirical source distribution	rwanda_public_transport_delays.csv
Generator	Samples rows and assigns new event identity	Python standard library
Producer	Publishes route-keyed JSON events	confluent-kafka
Operational store	Stores live trip events	MySQL / InnoDB
Dashboard	Summarizes and visualizes current state	Django + Django REST Framework
Historical analytics	Stores training input and model artifacts	HDFS + Spark MLlib

4. Data Generation and Event Model

TripEventGenerator samples a complete source row rather than sampling individual columns independently. This preserves useful relationships such as route and district combinations, weather and precipitation, congestion and delay outcomes, and transport type patterns.



Each event keeps its trip_id, route, schedule, location, weather, and delay attributes.

An event receives a new SIM-prefixed trip ID, current date and time, scheduled departure and arrival values based on the source duration, and an event_generated_at timestamp in GMT+2. Numeric CSV fields are converted before JSON serialization so Kafka consumers, Spark, and MySQL receive usable numeric types.

Representative event fields

Group	Fields
Identity	trip_id, date, time, event_generated_at
Network	route_id, vehicle_id, transport_type
Location	origin/destination station, district, latitude, longitude
Schedule and delay	scheduled times, actual departure/arrival delay, delayed
Conditions	weather, temperature, humidity, wind, precipitation
Demand and traffic	passengers carried, attendance estimate, congestion index, peak hour

5. Streaming and Reliability

The Django REST producer publishes each generated event to the configured Kafka topic, normally kigali-trip-events. The route_id is used as the Kafka key. Kafka's keyed partitioning keeps events for one route ordered while allowing independent routes to be processed in parallel.

The producer is configured with acknowledgements from all in-sync replicas and idempotence enabled. It flushes outstanding messages and returns delivery metadata only after the broker acknowledges the records. This gives the demonstration a visible delivery contract rather than silently assuming that a send succeeded.

Choice	Reason
route_id key	Preserve per-route ordering and distribute different routes.
acks=all	Wait for the strongest available broker acknowledgement.
enable.idempotence=true	Reduce duplicate writes caused by producer retries.
delivery callback	Collect partition and offset information for the API response.
manual consumer commits	Make at-least-once processing explicit and observable.

The project does not claim end-to-end exactly-once semantics. That trade-off is documented so the pipeline's guarantees remain understandable and defensible.

6. MySQL Operational Storage

MySQL is the single runtime database for the dashboard and generated events. The `transport_trip_events` table is created by the existing persistence helper when needed and uses an InnoDB table with indexes for generated time and route queries.

Storage concern	Implementation
Primary identity	<code>trip_id VARCHAR(64) PRIMARY KEY</code>
Time fields	DATE, TIME, and DATETIME columns
Numeric features	DECIMAL, INT, and TINYINT columns
Query support	Indexes on <code>event_generated_at</code> and <code>route_id</code>
Repeatable loads	ON DUPLICATE KEY UPDATE for existing trip IDs
Character set	utf8mb4

SQLite-to-MySQL migration

The project previously accumulated records while the dashboard used SQLite. A dedicated Django management command reads `dashboard/db.sqlite3`, normalizes timestamps, and copies rows to MySQL in batches. It can be rerun safely because the MySQL writer uses `trip_id` as the conflict key.

```
.\.venv\Scripts\python.exe .\dashboard\manage.py replicate_sqlite_to_mysql
```

After migration, the application is configured for MySQL-only runtime reads and writes. The old SQLite file is retained only as an archival migration source.

7. Django Dashboard and API

The dashboard is a Django application with REST endpoints for live data, summary metrics, event generation, and Kafka publishing. The browser refreshes current data on a short interval so a running demonstration visibly changes as new events arrive.

Endpoint	Function
/	Operational command-center dashboard.
/api/dashboard-data/	Recent events, summary metrics, route and risk information.
/api/database-summary/	Full-table aggregates and recent stored events.
/api/data/generate/	Generate a batch and persist it to MySQL.
/api/trips/publish/	Generate, store, and publish events to Kafka.
/api/preview-event/	Return one non-persisted preview event.

The dashboard combines observed values with an explainable feature-based risk estimate. The estimate uses congestion, precipitation, peak hour, and weather penalties to produce a probability and predicted delay indicator while the Spark model path is being trained or deployed.

Operational views

- KPI cards for stored events, delay rate, predicted delays, and top route.
- Leaflet map markers based on origin coordinates.
- Route volume bars and transport mix breakdown.
- Recent trip activity with route, mode, district, and risk state.
- Database summary showing full-table totals rather than only the latest window.

8. HDFS and Spark MLlib Workflow

The historical analytics path stores the reference CSV in HDFS and trains a delay classifier with Spark MLlib. This demonstrates the transition from an operational stream to distributed historical processing.

Stage	Output
HDFS setup	Project-owned NameNode and DataNode directories.
Reference upload	Historical CSV stored in HDFS.
Feature preparation	Planning and dispatch features assembled without target leakage.
Model training	Spark MLlib pipeline for delay classification.
Evaluation	AUC, accuracy, F1, and row counts.
Artifacts	/models/kigali_delay_classifier and /analytics/model_metrics

Avoiding target leakage

The training job deliberately excludes actual arrival and departure delay fields because those values are only known after a trip has occurred. Including them would make evaluation look stronger while producing a model that could not support planning-time decisions.

```
$env:HADOOP_CONF_DIR = "$PWD\infrastructure\hadoop-conf"
C:\Spark_set_up\spark-4.2.0\bin\spark-submit.cmd .\ml\training\train_delay_classifier.py
```

9. Testing and Validation

Validation focused on the highest-risk integration boundaries: database selection, event persistence, MySQL compatibility, and the dashboard generation endpoint.

Check	Result
Django system check	Passed with no reported issues.
Python compilation	Edited settings, views, migration command, and generator compiled successfully.
MySQL backend resolution	Django reported vendor=mysql and the configured database name.
Dashboard generation request	HTTP 201 with database_inserted count returned.
Live MySQL row verification	Generated row count increased after the request.
SQLite migration	Historical rows copied in batches with rerunnable upsert behavior.
Unit tests available	Summary and transport-means aggregation tests in tests/test_summary.py.

A MySQL-specific issue was found and corrected during integration: delayed is reserved in some MySQL contexts and must be quoted in aggregate SQL expressions. This illustrates why database-backed endpoint tests are important even when the generator itself succeeds.

10. Installation and Demonstration

The project is designed for local Windows development with services started in separate PowerShell terminals.

1. Install dependencies

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

2. Configure MySQL

```
$env:MYSQL_DATABASE = "Rwanda_Public_Transport"
$env:MYSQL_USER = "root"
$env:MYSQL_PASSWORD = "your-password"
$env:MYSQL_HOST = "localhost"
$env:MYSQL_PORT = "3306"
```

3. Start the dashboard

```
.\.venv\Scripts\python.exe .\dashboard\manage.py runserver
```

4. Generate or publish data

```
.\.venv\Scripts\python.exe .\generator\trip_generator.py --rate 5 --count 10 --mysql
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:8000/api/trips/publish/ -ContentType "application/json"
```

The dashboard is then available at <http://127.0.0.1:8000/>.

11. Limitations and Future Work

The current implementation is intentionally educational and locally deployable. Several areas would need hardening for production use.

- Move credentials and secrets to a managed secret store rather than local environment files.
- Use a production WSGI server, authentication, authorization, and HTTPS for the dashboard.
- Add a dedicated Kafka consumer or Kafka Connect JDBC sink with operational monitoring.
- Add schema versioning and data-quality validation for changing event contracts.
- Measure model performance on a held-out, time-aware validation set and monitor drift.
- Add integration tests that start or mock Kafka and MySQL services in a controlled environment.

Conclusion

This project demonstrates a complete, inspectable big data workflow: historical data becomes realistic events; events move through a keyed stream; MySQL provides operational persistence; Django exposes live decisions; and HDFS plus Spark extend the same domain into distributed analytics and machine learning.

The strongest practical outcome is the clear separation of concerns. Each component has a defined responsibility, a documented interface, and a validation path, making the platform suitable for demonstration, assessment, and future extension.

12. Appendix: Repository Map

Path	Role
generator/trip_generator.py	Reference-driven synthetic event generation and direct MySQL mode.
producer/kafka_publisher.py	Route-keyed Kafka publishing with delivery acknowledgement.
dashboard/config/settings.py	Environment loading and MySQL-only Django database configuration.
dashboard/operations/views.py	Persistence, dashboard APIs, summaries, and risk estimation.
dashboard/operations/templates/operations/dashboard.html	Live operational dashboard UI.
dashboard/operations/management/commands/replicate_sqlite_to_mysql.py	Historical SQLite-to-MySQL migration.
ml/training/train_delay_classifier.py	Spark MLlib training job.
infrastructure/	Kafka, HDFS, and local Windows setup scripts.
tests/test_summary.py	Summary and transport aggregation tests.

End of report